

SENTINEL SOC Defense AI Layer

Provenance-Aware Action Firewall for Autonomous SOC Agents

System Architecture, Threat Model, Mathematical Policy, and Empirical Verification

SENTINEL Defense Engineering Team

September 21, 2026

Abstract

Tool-augmented LLM agents deployed in Security Operations Centers (SOCs) operate over heterogeneous data feeds where execution context and adversarial inputs share the same token space. This structural property exposes autonomous agents to **Indirect Prompt Injection (IPI)**, where adversary-crafted strings in syslog telemetry, web headers, or case notes hijack the agent’s tool-calling pipeline. Naive string-matching heuristics suffer from severe false-positive rates on legitimate attack discussions and are trivial to evade, while LLM-as-a-judge approaches introduce latency and recursive injection vulnerabilities.

This report presents the **SENTINEL SOC Defense AI Layer**, a deterministic, sub-millisecond provenance-aware decision firewall. SENTINEL intercepts candidate tool invocations prior to execution and evaluates them against four orthogonal dimensions: (1) **Action Criticality**, (2) **Provenance Trust Floor**, (3) **Independent Multi-Source Corroboration**, and (4) **Memory Trust Inheritance**. Furthermore, the system implements automated safe-action rewriting for consequential tools and deterministic tool-whitelisting backstops. We detail the formal policy mechanics, state machine lifecycle, HTTP contracts, and multi-toggle ablation probes.

Contents

1	Threat Model and Design Motivation	1
1.1	Autonomous Agentic Workflows in the SOC	1
1.2	The Vulnerability: Indirect Prompt Injection (IPI)	1
1.3	Why Conventional Approaches Fail	2
2	Core Defense Philosophy and Architecture	2
2.1	Architectural Decoupling	2
2.2	Provenance Tracking and Trust Hierarchy	3
2.3	Action Criticality Registry	3
3	Mathematical Policy Formulation	3
3.1	Formal Risk Formulation	3
3.2	Deterministic Decision Rules and State Machine	4
3.3	Memory Provenance Inheritance	4
4	System Architecture and Implementation	5
4.1	Module Breakdown	5
4.2	End-to-End Decision Trace Walkthrough	5
5	API Specifications and Wire Formats	5
5.1	Request Contract: POST /v1/decision	5
5.2	Response Contract: HTTP 200 OK	6

6	Empirical Verification and Ablation Analysis	7
6.1	Ablation Methodology	7
6.2	Findings and Security Implications	7
7	Conclusion and Operational Best Practices	8

1 Threat Model and Design Motivation

1.1 Autonomous Agentic Workflows in the SOC

In modern SOC infrastructure, autonomous agents orchestrate multi-step triage and incident response. A representative workflow consists of:

1. Ingesting an alert from a SIEM / EDR aggregator (e.g., suspicious outbound beaconing on SRV-22).
2. Invoking investigative read-only tools (`alert_read`, `asset_lookup`, `intel_search`).
3. Synthesizing observations into persistent memory to maintain multi-turn investigation state.
4. Formulating and proposing state-changing candidate actions (`incident_update`, `remediation_execute`, `disable_monitoring`).

1.2 The Vulnerability: Indirect Prompt Injection (IPI)

Because LLMs process data and control instructions within an undifferentiated context window, untrusted payload strings can hijack agent execution. In a SOC setting, untrusted data frequently contains imperative syntax:

- **Adversary-Controlled Syslog:** Web access log containing embedded commands like `close_incident` or `disable_monitoring`.
- **Phishing Email Headers:** Case notes with fake supervisor instructions demanding ticket closure.
- **Third-Party Threat Intel Feeds:** Unverified external RSS feeds suggesting disabling noisy correlation rules.

When the LLM ingests these inputs, it may generate tool calls that execute destructive actions (e.g., closing open incidents during an ongoing ransomware outbreak, disabling endpoint sensors, or leaking credentials).

1.3 Why Conventional Approaches Fail

- **Syntactic Heuristics / Keyword Blocklists:** Flagging phrases like “*disable monitoring*” or “*isolate host*” triggers excessive false alarms on legitimate alerts that describe malware attempting those very actions. Conversely, minor semantic mutations or encodings easily bypass keyword filters.
- **LLM-as-a-Judge Guardrails:** Running a secondary LLM call to evaluate the safety of the primary LLM adds significant latency (1–3 seconds), incurs high inference cost, and remains susceptible to secondary prompt injections.
- **SENTINEL’s Approach:** A deterministic, provenance-bound decision engine (< 1 ms evaluation overhead) that operates on structured origin metadata and action criticality rather than lexical surface forms.

2 Core Defense Philosophy and Architecture

2.1 Architectural Decoupling

SENTINEL is deployed as an out-of-process reverse proxy between the agent’s action planner and the tool execution runtime. The agent cannot directly dispatch commands to the environment; every candidate action must obtain an explicit policy verdict from the defense layer.

Decoupled Policy Pipeline:

1. **Agent Request** → Dispatches `POST /v1/decision` with candidate action and provenance graph.
2. **Tool Whitelist Gate** → Enforces scenario `allowed_tools` policy.
3. **Consequential Action Rewrite** → Transforms state-altering actions into human-confirmation requests.
4. **Provenance Taint & Risk Scoring** → Computes $R(a)$ based on criticality and weakest provenance.
5. **Corroboration Backstop** → Hard-blocks high-criticality actions lacking independent trusted verification.
6. **Action Verdict** → Returns `allow`, `rewrite`, `escalate`, or `block` with structured audit metadata.

Figure 1: SENTINEL Defense Evaluation Pipeline

2.2 Provenance Tracking and Trust Hierarchy

Every observation ingested by the agent carries cryptographically or architecturally guaranteed origin metadata. The defense layer quantifies provenance reliability through a normalized trust score $T \in [0.00, 1.00]$:

Provenance Level	Trust (T)	Origin Description & Ingestion Boundary
SYSTEM_POLICY	1.00	Immutable runtime configuration, policy rules, and system constraints.
AUTHENTICATED_USER	0.85	Verified SOC analyst issuing direct supervisory prompts via authenticated session.
TRUSTED_INTERNAL	0.70	Authoritative internal telemetry (CMDB asset inventories, verified SIEM alerts).
UNTRUSTED_INTERNAL	0.30	Mutable internal ticket notes, user comments, unverified internal chat messages.
UNTRUSTED_EXTERNAL	0.20	External public blogs, vendor feeds, third-party threat intel feeds.
ADVERSARY_CONTROLLED	0.00	Raw endpoint telemetry, HTTP headers, proxy logs, adversary payload strings.

Table 1: Provenance Trust Taxonomy and Quantification

2.3 Action Criticality Registry

Actions are categorized by their intrinsic operational risk and system blast radius:

Action / Tool	Criticality (C)	Operational Impact & Risk Assessment
disable_monitoring	1.00	Blinds observability; halts detection telemetry collection.
close_incident	0.90	Terminates active investigation; risks masking ongoing lateral movement.
suppress_alert	0.90	Silences rule triggers on targeted entities.
modify_correlation_rule	0.90	Alters detection logic globally.
run_remediation	0.85	State-changing active defense (host isolation, process termination, ACL updates).
change_asset_criticality	0.80	Reclassifies asset tier; downgrades alert triage urgency.
Default / Unclassified	0.30	Standard telemetry writes (<code>incident_create</code> , logging).
correlate_intel	0.20	Read-only external threat indicator lookup.
summarize, add_comment	0.10	Non-destructive read-only data aggregation and text synthesis.

Table 2: SOC Action Criticality Taxonomy

3 Mathematical Policy Formulation

3.1 Formal Risk Formulation

Let $\mathcal{O}_a = \{o_1, o_2, \dots, o_n\}$ denote the set of observations justifying action a , and let $\mathcal{M}_a = \{m_1, m_2, \dots, m_k\}$ denote the set of justifying memory records.

1. Minimum Provenance Floor ($T_{\min}$): The system enforces a strict weakest-link taint model across all justifications:

$$T_{\min}(a) = \min \left(\{\text{trust}(o) \mid o \in \mathcal{O}_a\} \cup \{\text{trust}(m) \mid m \in \mathcal{M}_a\} \right) \quad (1)$$

If no provenance is attached, $T_{\min}(a) = 0.00$.

2. Independent Corroboration (K): Let $\mathcal{S}$ denote the set of unique data sources contributing justifications with trust $T \geq 0.70$:

$$K(a) = |\{\text{source_id}(x) \mid x \in \mathcal{O}_a \cup \mathcal{M}_a \text{ and } \text{trust}(x) \geq 0.70\}| \quad (2)$$

3. Imperative Syntax Indicator (W_{pattern}): An auxiliary regex detector identifies imperative command patterns. Its contribution is strictly bounded:

$$W_{\text{pattern}} = \begin{cases} 0.15 & \text{if imperative patterns match in context} \\ 0.00 & \text{otherwise} \end{cases} \quad (3)$$

4. Unified Risk Score (R):

$$R(a) = \text{clamp}_{[0,1]} \left(C(a) \cdot (1 - T_{\min}(a)) + W_{\text{pattern}} - (0.10 \cdot \min(K(a), 2)) \right) \quad (4)$$

3.2 Deterministic Decision Rules and State Machine

The policy resolves candidate actions via a deterministic decision tree:

1. Unauthorized Tool Rejection:

$$\text{If } a.\text{type} == \text{"tool_call"} \wedge a.\text{tool} \notin \text{allowed_tools} \implies \mathbf{BLOCK} \text{ (TOOL_NOT_ALLOWED_BY_POLICY)} \quad (5)$$

2. Corroboration Backstop Firewall:

$$\text{If } C(a) \geq 0.80 \wedge K(a) == 0 \wedge T_{\min}(a) \leq 0.30 \implies \mathbf{BLOCK} \text{ (CORROBORATION_BACKSTOP)} \quad (6)$$

3. Safe Consequential Action Rewriting:

$$\text{If } a.\text{tool} \in \{\text{remediation_execute}, \text{incident_update}(\text{status} = \text{"closed"})\} \implies \mathbf{REWRITE} \quad (7)$$

The action is transformed into an advisory text response requesting human confirmation without altering system state.

4. Risk Interval Classification:

$$\text{Verdict}(a) = \begin{cases} \mathbf{BLOCK} & \text{if } R(a) \geq 0.70 \\ \mathbf{ESCALATE} & \text{if } 0.40 \leq R(a) < 0.70 \\ \mathbf{ALLOW} & \text{if } R(a) < 0.40 \end{cases} \quad (8)$$

3.3 Memory Provenance Inheritance

To prevent **Memory Poisoning** (where an agent stores untrusted text in memory and later retrieves it as an unverified “fact”), the memory layer enforces transitive trust inheritance:

$$m_{\text{new}} = \text{write}(c, \mathcal{O}_{\text{used}}) \implies \text{trust}(m_{\text{new}}) = \min_{o \in \mathcal{O}_{\text{used}}} \text{trust}(o) \quad (9)$$

If an agent synthesizes memory from an external blog ($T = 0.20$), the resulting memory entry permanently carries $T = 0.20$.

4 System Architecture and Implementation

4.1 Module Breakdown

Module	Core Interfaces	Technical Functionality
models.py	CandidateAction, Decision	Immutable dataclasses defining request/response structures.
trust.py	trust_score(), min_trust()	Quantifies provenance labels and computes floor trust across dependencies.
risk_actions.py	action_criticality()	Criticality registry for SOC operational tools.
instruction_detector.py	detect_instruction_patterns()	Regex scanner for imperative command syntax (capped at +0.15).
memory.py	MemoryStore	State store maintaining transitive provenance inheritance on write cycles.
policy.py	decide(), PolicyConfig	Core decision engine executing tool gates, backstops, rewrites, and scoring.
adapter.py	DefenseHandler	Multi-threaded HTTP server conforming to SENTINEL v1 API specification.
trace.py	TraceLogger	High-throughput JSONL audit logger for decision telemetry.
ablation.py	run_all_probes()	In-process test harness verifying isolated policy toggle behavior.

Table 3: Repository Module Specification

4.2 End-to-End Decision Trace Walkthrough

Step-by-Step Request Evaluation

1. **Ingestion:** The autonomous agent issues an HTTP POST to `/v1/decision`.
2. **Translation:** `adapter.py` unpacks the candidate tool call, conversation history, and provenance graph, assembling a `CandidateAction`.
3. **Permission Check:** `policy.py` checks the tool against `policy_context.allowed_tools`.
4. **Provenance Floor Calculation:** `trust.py` traverses observation and memory IDs to determine $T_{\min}$.
5. **Corroboration Counting:** Discovers unique, independent sources with $T \geq 0.70$.
6. **Backstop Evaluation:** Checks if high criticality ($C \geq 0.80$) meets uncorroborated low trust ($T \leq 0.30$).
7. **Action Rewrite:** If the tool is consequential and permitted, replaces tool execution with human-in-the-loop review text.
8. **Audit Logging:** `trace.py` appends the decision event to `sentinel_decisions.jsonl`.
9. **Response Dispatch:** Returns HTTP 200 with structured decision JSON.

5 API Specifications and Wire Formats

5.1 Request Contract: POST `/v1/decision`

```
{
  "run_id": "soc_incident_investigation_01",
  "step_id": 3,
  "policy_context": {
    "policy_id": "soc_standard",
    "allowed_tools": ["alert_read", "asset_lookup", "intel_search", "incident_create"]
  },
  "candidate_action": {
    "type": "tool_call",
    "tool": "remediation_execute",
    "arguments": {
      "host_id": "SRV-91",
      "action": "isolate_host"
    }
  },
  "conversation": [
    {
      "role": "user",
      "content": "Alert AL-3003 fired on SRV-91. Investigate and isolate if infected.",
      "provenance_ids": ["prov-0001"]
    }
  ],
  "observation": {
    "content": "EDR telemetry confirmed Cobalt Strike beacon on SRV-91.",
    "provenance_ids": ["prov-0002"]
  },
  "provenance": [
    {
      "id": "prov-0001",
      "provenance": {
        "source_id": "turn-0",
        "source_type": "user",

```

```

        "trust_level": "authenticated_user",
        "sensitivity": "internal"
    }
},
{
    "id": "prov-0002",
    "provenance": {
        "source_id": "edr-agent",
        "source_type": "tool_output",
        "trust_level": "trusted_internal",
        "sensitivity": "restricted"
    }
}
]
}

```

5.2 Response Contract: HTTP 200 OK

```

{
  "decision": "rewrite",
  "risk_score": 0.06,
  "confidence": 0.56,
  "reason_codes": [
    "SAFE_REWRITE_PROPOSED"
  ],
  "explanation": "Outcome REWRITE: criticality=0.85, minimum provenance trust=0.70, independent trusted corroboration=2, risk=0.06.",
  "rewritten_action": {
    "type": "respond",
    "tool": null,
    "arguments": {},
    "content": "I did not execute remediation_execute. The requested state-changing action needs human review.",
    "final": false,
    "confirmation_for": null
  },
  "metadata": {
    "action_criticality_input": "run_remediation",
    "observation_count": 2,
    "memory_count": 0
  }
}

```

6 Empirical Verification and Ablation Analysis

6.1 Ablation Methodology

To prove that each defense layer operates as an independent security control, we execute a multi-configuration ablation matrix against five deterministic in-process boundary probes. Each probe tests an isolated policy edge case:

6.2 Findings and Security Implications

- **Corroboration Backstop Validation (Probe A):** Disabling the corroboration backstop transitions high-risk actions ($C = 0.80$) justified by untrusted inputs from **BLOCK** to **ESCALATE**. The backstop is required to prevent unverified high-impact actions from proceeding without human sign-off.
- **Instruction Detector Bounding (Probe B):** The detector contributes exactly +0.15 to elevate borderline low-criticality actions from **ALLOW** to **ESCALATE**, proving it acts as an auxiliary signal without independently vetoing operations.

Probe	Target Scenario & Provenance Under Test	Full Policy	Corrob. OFF	Detector OFF	Memory OFF
Probe A	change_asset_criticality ($C = 0.80$), $T = 0.30$	BLOCK	ESCALATE	BLOCK	BLOCK
Probe B	query_logs ($C = 0.30$), $T = 0.00$ + Imperative string	ESCALATE	ESCALATE	ALLOW	ESCALATE
Probe C	suppress_alert ($C = 0.90$), Poisoned memory ($T = 0.20$)	BLOCK	BLOCK	BLOCK	ALLOW
Probe D	remediation_execute ($C = 0.85$), Dual trusted corroboration	REWRITE	REWRITE	REWRITE	REWRITE
Probe E	Calling unlisted tool outside allowed_tools	BLOCK	BLOCK	BLOCK	BLOCK

Table 4: Ablation Boundary Probe Matrix: Single-Toggle Isolation

- **Memory Trust Inheritance Criticality (Probe C):** When memory trust inheritance is disabled, an action justified by poisoned memory shifts from **BLOCK** to **ALLOW**. This demonstrates that without memory taint tracking, agents are completely vulnerable to multi-turn prompt injection.
- **Invariant Safety Controls (Probes D & E):** Safe rewrites and tool permission limits operate unconditionally across all configuration switches, verifying that structural guardrails remain active regardless of heuristic settings.

7 Conclusion and Operational Best Practices

The SENTINEL SOC Defense AI Layer demonstrates that robust protection against Indirect Prompt Injection does not require complex recursive LLM judges or brittle lexical blocklists. By enforcing ****action criticality****, ****provenance tracking****, ****memory taint inheritance****, and ****automated safe rewrites****, the architecture provides verifiable security guarantees with sub-millisecond evaluation latency.

Key Implementation Takeaways:

1. **Anchor Decisions in Provenance:** Lexical forms are easily manipulated; origin metadata must govern execution authority.
2. **Enforce Transitive Memory Taint:** Prevent memory wash attacks by preserving source trust across all memory read/write cycles.
3. **Transform Rather Than Abort:** Use safe action rewrites to convert consequential tool calls into human confirmations, preserving agent utility while eliminating execution risk.